SE EXPERIMENT 10

Aim: To perform software testing (white-box testing) on a core module of the Smart Parking Management System (SPMS).

Theory:

White box testing (also referred to as glass box testing) evaluates the internal coding and infrastructure of software components by analysing predefined inputs against expected outputs. It focuses on inner execution pathways, conditional loops, boundary conditions, and code paths to expose hidden security flaws or broken logical structures.

White Box Testing Evaluates:

- Internal structural bugs and security vulnerabilities.
- Broken, poorly structured, or dead branches in coding logic.
- Flow of variable states and parameter inputs through the execution graph.
- Functionality and termination conditions of iterative loops.
- Statement, branch, and condition coverage on a unit/functional level.

Flow Graph Notation:

A flow graph is a directed graph $G = (V, E)$ where nodes (N) represent procedural statements or sequential basic blocks, and directed edges (E) model control transfers between statements. A predicate node (P) is a decision branch containing a condition that splits the control flow into two or more distinct paths. Enclosed bounded areas and open exterior spaces constitute non-overlapping regions (R).

Cyclomatic Complexity:

Cyclomatic complexity, $V(G)$, quantifies the logical complexity of source code and defines the exact upper bound of linearly independent execution paths required to achieve complete branch coverage. It is calculated using three standard methods:

- Method 1: $V(G) = P + 1$, where P is the count of predicate nodes.
- Method 2: $V(G) = E - N + 2$, where E is the number of directed edges and N is the total count of nodes.
- Method 3: $V(G) =$ Total number of non-overlapping regions (R) in graph G.

Advantages of White Box Testing:

- Enables precise code optimization by locating logical anomalies and dead blocks early.
- Test suites can be integrated directly into automated unit testing pipelines.
- Provides high structural confidence by verifying uncovered branches.
- Can begin implementation before external user interfaces or hardware connections are finalized.

Disadvantages of White Box Testing:

- Designing detailed white box test suites requires programming competence and code access.
- Time-intensive and complex for extensive production codebases.
- Overlooking real-world user flow variants if developer testing lacks comprehensive edge-case scenarios.

Sample Code: Gate Entry QR Verification & Barrier Control Module

The selected code implements core functional requirements from Section QR Code Entry & Exit Gate Processing and Section Fail-Safe Gate Barrier Safety SR-1 of the SPMS:

```java
public class GateController {
    public String verifyAndTriggerEntry(String qrToken, boolean isHardwareHealthy, boolean isSlotAvailable) {
        // Node 1: Entry and Initial check
        if (!isHardwareHealthy) { // Predicate Node 2
            return "FAILSAFE_MANUAL_MODE"; // Node 3
        }

        // Node 4: Decode token & check validity
        boolean isTokenValid = validateTokenSignature(qrToken);
        if (!isTokenValid) { // Predicate Node 5
            return "ACCESS_DENIED_INVALID_QR"; // Node 6
        }

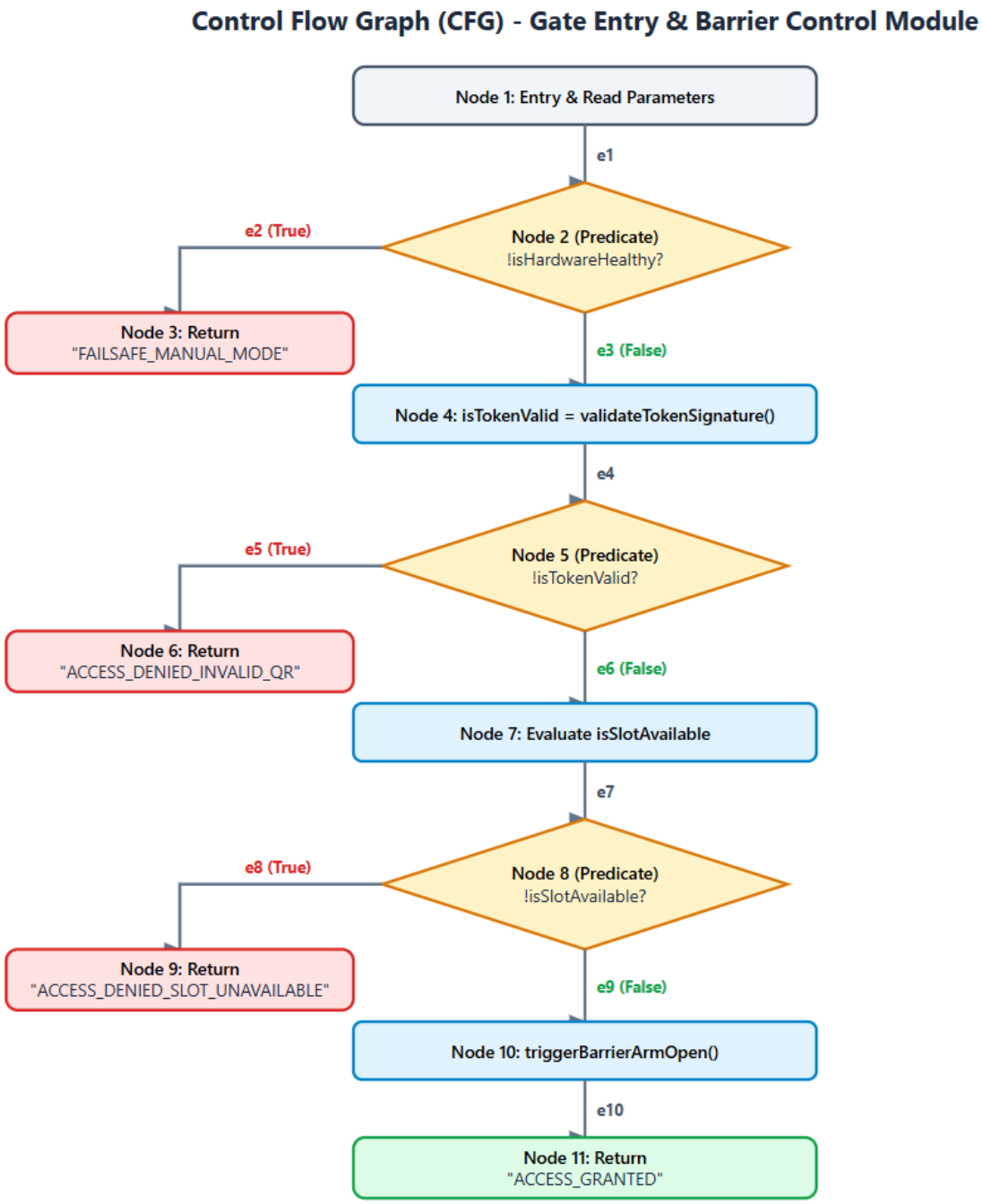
        // Node 7: Evaluate slot readiness
        if (!isSlotAvailable) { // Predicate Node 8
            return "ACCESS_DENIED_SLOT_UNAVAILABLE"; // Node 9
        }

        // Node 10: Successful validation and gate arm activation
        triggerBarrierArmOpen(); // Node 10
        return "ACCESS_GRANTED"; // Node 11
    }

    private boolean validateTokenSignature(String token) {
        return token != null && token.startsWith("SPMS-VALID");
    }

    private void triggerBarrierArmOpen() {
        // Sends MQTT open signal to ESP32 barrier relay
    }
}
```

Control Flow Representation



Calculation of Cyclomatic Complexity, $V(G)$

- Method 1 (Predicate Nodes):
  - Number of predicate nodes (P) = 3 (Nodes 2, 5, 8)
  - $V(G) = P + 1 = 3 + 1 = 4$
- Method 2 (Edges and Nodes Formula):
  - Number of directed edges (E) = 10
  - Number of nodes (N) = 8 (consolidating distinct basic-block operational and terminal nodes: 1, 2, 3, 4-5, 6, 7-8, 9, 10-11)
  - $V(G) = E - N + 2 = 10 - 8 + 2 = 4$

- Method 3 (Non-overlapping Regions):
  - Bounded closed regions (R_1, R_2, R_3) + 1 unbounded external region (R_4)
  - $V(G) = R = 4$

Independent Path Test Cases Design

Parameter / Field	Test Case 1	Test Case 2	Test Case 3	Test Case 4
Test Case ID	TC-WB-01	TC-WB-02	TC-WB-03	TC-WB-04
Target Independent Path	$1 \rightarrow 2 \rightarrow 3$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 11$
Input: QR_Token	"SPMS-VALID-99"	"INVALID-EXPIRED-TOKEN"	"SPMS-VALID-102"	"SPMS-VALID-205"
Input: isHardwareHealthy	false	true	true	true
Input: isSlotAvailable	true	true	false	true
Expected Output	"FAILSAFE_MANUAL_MODE"	"ACCESS_DENIED_INVALID_QR"	"ACCESS_DENIED_SLOT_UNAVAILABLE"	"ACCESS_GRANTED"
Execution Status	Pass	Pass	Pass	Pass

Conclusion:

White-box software testing was studied and successfully executed on the Smart Parking Management System entry and barrier control module. The control flow graph was constructed, and the Cyclomatic Complexity was evaluated to be 4 across all three analytical formulations. Four linearly independent paths were isolated, and test cases were defined to achieve 100% statement and branch test coverage.